

Full Adder using Two Half Adders

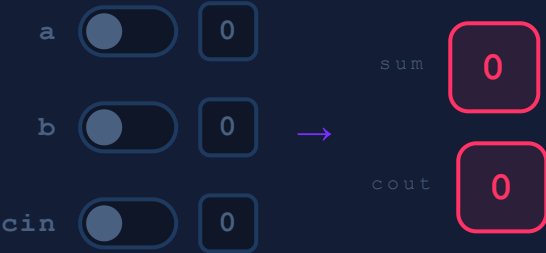
STRUCTURAL

entity: full_adder

$$\text{sum} = a \oplus b \oplus \text{cin} \quad | \quad \text{cout} = (a \cdot b) + (\text{cin} \cdot (a \oplus b))$$

Adds three 1-bit numbers using TWO half_adder components plus one OR gate. HA1 adds a,b → sum1 + carry1. HA2 adds sum1,cin → final sum + carry2. The two carries are OR'd for cout. Pure structural (component instantiation) architecture.

LIVE SIMULATOR



$$\text{sum} = a \text{ XOR } b \text{ XOR } \text{cin} \quad | \quad \text{cout} = c1 \text{ OR } c2$$

TRUTH TABLE

a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

► VHDL - FULL_ADDER.VHD (STRUCTURAL)

COPY

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
    Port (
        a      : in  STD_LOGIC;
        b      : in  STD_LOGIC;
        cin    : in  STD_LOGIC;
        sum     : out STD_LOGIC;
        cout    : out STD_LOGIC
    );
end full_adder;

architecture Structural of full_adder is

    component half_adder is
        Port (a : in  STD_LOGIC;
              b : in  STD_LOGIC;
              s : out STD_LOGIC;
              c : out STD_LOGIC);
    end component;

    signal s1, c1, c2 : STD_LOGIC;

begin
    HA1: half_adder port map(a, b, s1, c1);
    HA2: half_adder port map(s1, cin, sum, c2);
    cout <= c1 or c2;
end Structural;
```

► SUB-COMPONENT - HALF_ADDER.VHD (MUST COMPILE FIRST)

[COPY](#)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

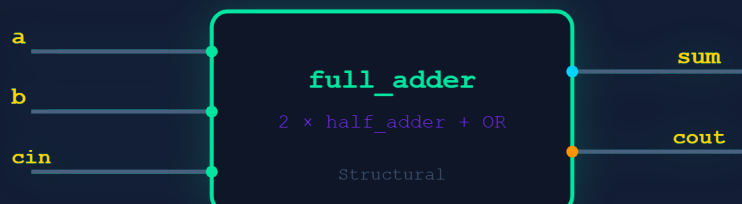
```
entity half_adder is
    Port (
        a : in  STD_LOGIC;
        b : in  STD_LOGIC;
        s : out STD_LOGIC;
        c : out STD_LOGIC
    );
end half_adder;

architecture Behavioral of half_adder is
begin
    s <= a xor b;
    c <= a and b;
end Behavioral;
```

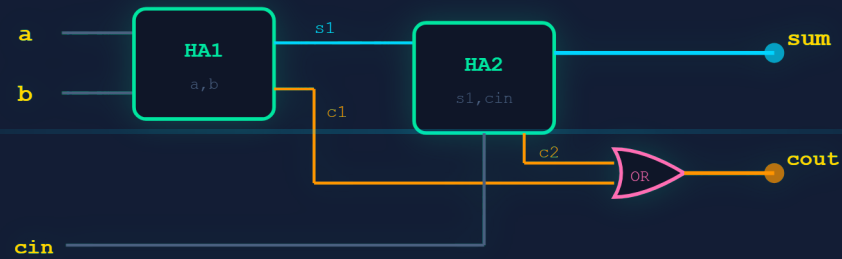
```
HA1(a,b) → s1 = a⊕b, c1 = a·b
HA2(s1,cin) → sum = s1⊕cin, c2 = s1·cin
OR → cout = c1 + c2
```

COMPILE ORDER: 1. half_adder.vhd → 2. full_adder.vhd

► BLOCK DIAGRAM - FULL_ADDER



► STRUCTURAL CIRCUIT - HA1, HA2, OR



```
HA1:a,b→s1,c1 | HA2:s1,cin→sum,c2 | cout=c1 OR c2
```

► TESTBENCH - FULL_ADDER_TB.VHD

COPY

```

ENTITY full_adder_tb IS END full_adder_tb;
ARCHITECTURE behavior OF full_adder_tb IS
    COMPONENT full_adder
        PORT(a,b,cin : IN  std_logic;
             sum,cout : OUT std_logic);
    END COMPONENT;
    signal a,b,cin : std_logic := '0';
    signal sum,cout : std_logic;
BEGIN
    uut: full_adder PORT MAP(a,b,cin,sum,cout);
    process begin
        a<='0';b<='0';cin<='0';wait for 100 ns;
        a<='0';b<='0';cin<='1';wait for 100 ns;
        a<='0';b<='1';cin<='0';wait for 100 ns;
        a<='0';b<='1';cin<='1';wait for 100 ns;
        a<='1';b<='0';cin<='0';wait for 100 ns;
        a<='1';b<='0';cin<='1';wait for 100 ns;
        a<='1';b<='1';cin<='0';wait for 100 ns;
        a<='1';b<='1';cin<='1';wait for 100 ns;
        wait;
    end process;
END;
```

► EXECUTION STEPS - XILINX ISE

1

New Project

→ File → New Project → full_adder_2ha , HDL: VHDL

2

Create half_adder

→ New Source → VHDL Module → half_adder (a,b,s,c)

3

Enter HA code

→ `s<=a xor b; c<=a and b;`

4

Create full_adder

→ `component half_adder + signals s1,c1,c2`

5

Port maps

→ `HA1(a,b,s1,c1); HA2(s1,cin,sum,c2); cout<=c1 or c2`

6

Check Syntax

→ Synthesize-XST → 0 errors

7

Testbench

→ VHDL Test Bench → all 8 input combos

8

Simulate

→ ISim → Run 800 ns → verify sum/cout

EXAM NOTE: STRUCTURAL – full adder instantiates half_adder TWICE + OR gate.
half_adder.vhd compiled before full_adder.vhd.

Full Adder using Half Adder

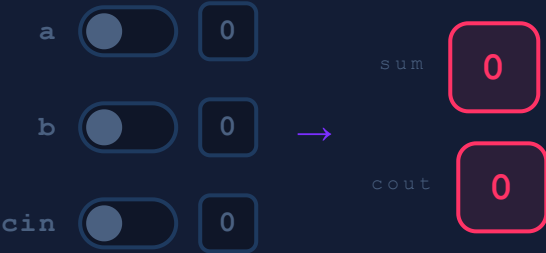
BEHAVIORAL

entity: full_adder_bh

$$\text{sum} = a \oplus b \oplus \text{cin} \quad | \quad \text{cout} = ab + b\text{cin} + a\text{cin}$$

Same full adder logic derived from the half-adder equations, written in a single behavioral/dataflow file – direct concurrent signal assignments, no component instantiation. sum reuses the half-adder XOR; cout is the majority function.

LIVE SIMULATOR



$$\text{sum} = a \text{ XOR } b \text{ XOR } \text{cin} \quad | \quad \text{cout} = (a \text{ AND } b) \text{ OR } (b \text{ AND } \text{cin}) \text{ OR } (a \text{ AND } \text{cin})$$

TRUTH TABLE

a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

► VHDL - FULL_ADDER_BH.VHD (BEHAVIORAL)

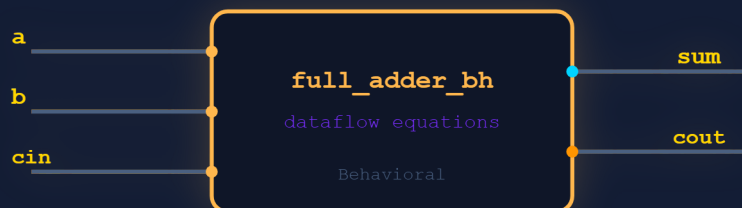
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder_bh is
    Port (
        a      : in  STD_LOGIC;
        b      : in  STD_LOGIC;
        cin     : in  STD_LOGIC;
        sum     : out STD_LOGIC;
        cout    : out STD_LOGIC
    );
end full_adder_bh;

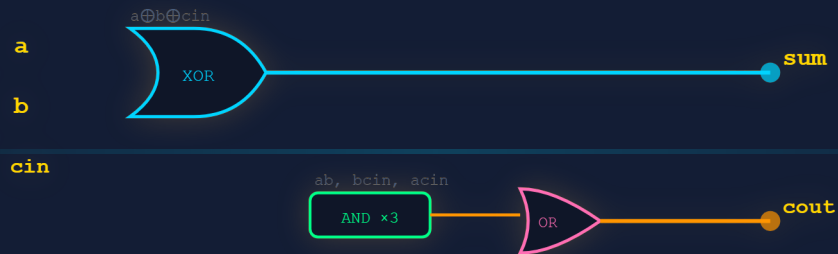
architecture Behavioral of full_adder_bh is
begin
    sum  <= a xor b xor cin;
    cout <= (a and b) or (b and cin) or (a and cin);
end Behavioral;
```

[COPY](#)

► BLOCK DIAGRAM - FULL_ADDER_BH



► GATE-LEVEL CIRCUIT



sum = 3-input XOR | cout = OR of 3 AND terms (majority)

► TESTBENCH - FULL_ADDER_BH_TB.VHD

```
ENTITY full_adder_bh_tb IS END full_adder_bh_tb;
ARCHITECTURE behavior OF full_adder_bh_tb IS
    COMPONENT full_adder_bh
        PORT(a,b,cin : IN  std_logic;
             sum,cout : OUT std_logic);
    END COMPONENT;
    signal a,b,cin : std_logic := '0';
    signal sum,cout : std_logic;
BEGIN
    uut: full_adder_bh PORT MAP(a,b,cin,sum,cout);
    process begin
        a<='0';b<='0';cin<='0';wait for 100 ns;
        a<='0';b<='0';cin<='1';wait for 100 ns;
        a<='0';b<='1';cin<='0';wait for 100 ns;
        a<='0';b<='1';cin<='1';wait for 100 ns;
        a<='1';b<='0';cin<='0';wait for 100 ns;
        a<='1';b<='0';cin<='1';wait for 100 ns;
        a<='1';b<='1';cin<='0';wait for 100 ns;
        a<='1';b<='1';cin<='1';wait for 100 ns;
        wait;
    end process;
END;
```

COPY

► EXECUTION STEPS - XILINX ISE

1

New Project

→ File → New Project → full_adder_bh , HDL: VHDL

2

New Source

→ VHDL Module → full_adder_bh – ports a,b,cin,sum,cout

3

Enter Equations

→ `sum<=a xor b xor cin;` and `cout majority`

4

Check Syntax

→ Synthesize-XST → 0 errors

5

Testbench

→ VHDL Test Bench → all 8 combos, 100 ns each

6

Simulate

→ ISim → Run 800 ns → verify

EXAM NOTE: BEHAVIORAL – single file, no components. Concurrent assignments only. Same truth table as the 2-HA structural version.

Full Subtractor using Two Half Subtractors

STRUCTURAL entity: full_subtractor

$diff = a \oplus b \oplus bin \quad | \quad bout = a'b + a'bin + b \cdot bin$

Subtracts with borrow-in (a - b - bin) using TWO half_subtractor components plus one OR gate. HS1: a,b → d1 + bo1. HS2: d1,bin → diff + bo2. The two borrows are OR'd for bout.

LIVE SIMULATOR

a

☐

0

b

☐

0

bin

☐

0

→

diff

0

bout

0

$diff = a \text{ XOR } b \text{ XOR } bin \quad | \quad bout = bo1 \text{ OR } bo2$

TRUTH TABLE

a	b	bin	diff	bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

VHDL - FULL_SUBTRACTOR.VHD (STRUCTURAL)

[COPY](#)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_subtractor is
    Port (
        a      : in  STD_LOGIC;
        b      : in  STD_LOGIC;
        bin    : in  STD_LOGIC;
        diff   : out STD_LOGIC;
        bout   : out STD_LOGIC
    );
end full_subtractor;

architecture Structural of full_subtractor is

    component half_subtractor is
        Port (a : in  STD_LOGIC;
              b : in  STD_LOGIC;
              d : out STD_LOGIC;
              bo : out STD_LOGIC);
    end component;

    signal d1, bo1, bo2 : STD_LOGIC;

begin
    HS1: half_subtractor port map(a, b, d1, bo1);
    HS2: half_subtractor port map(d1, bin, diff, bo2);
    bout <= bo1 or bo2;
end Structural;
```

► SUB-COMPONENT - HALF_SUBTRACTOR.VHD (MUST COMPILE FIRST)

[COPY](#)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

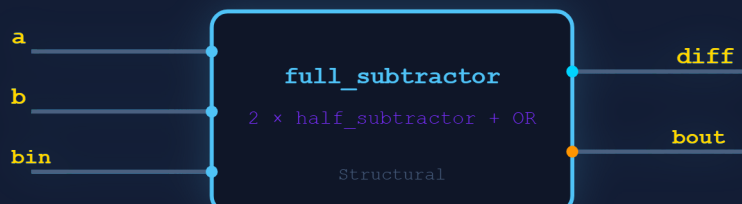
entity half_subtractor is
    Port (
        a : in  STD_LOGIC;
        b : in  STD_LOGIC;
        d : out STD_LOGIC;
        bo : out STD_LOGIC
    );
end half_subtractor;

architecture Behavioral of half_subtractor is
begin
    d <= a xor b;
    bo <= (not a) and b;
end Behavioral;
```

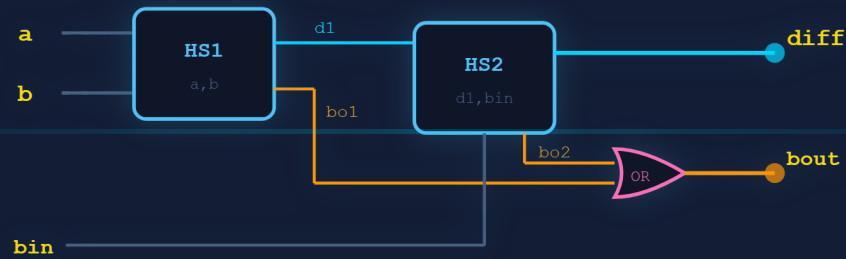
HS1 (a,b) $\rightarrow d1 = a \oplus b, bo1 = a' \cdot b$
HS2 (d1,bin) $\rightarrow diff = d1 \oplus bin, bo2 = d1' \cdot bin$
OR $\rightarrow bout = bo1 + bo2$

COMPILE ORDER: 1. half_subtractor.vhd $\rightarrow$ 2. full_subtractor.vhd

► BLOCK DIAGRAM - FULL_SUBTRACTOR



► STRUCTURAL CIRCUIT - HS1, HS2, OR



HS1:a,b→d1,bo1 | HS2:d1,bin→diff,bo2 | bout=bo1 OR bo2

► TESTBENCH - FULL_SUBTRACTOR_TB.VHD

COPY

```

ENTITY full_subtractor_tb IS END full_subtractor_tb;
ARCHITECTURE behavior OF full_subtractor_tb IS
    COMPONENT full_subtractor
        PORT(a,b,bin : IN  std_logic;
            diff,bout : OUT std_logic);
    END COMPONENT;
    signal a,b,bin : std_logic := '0';
    signal diff,bout : std_logic;
BEGIN
    uut: full_subtractor PORT MAP(a,b,bin,diff,bout);
    process begin
        a<='0';b<='0';bin<='0';wait for 100 ns;
        a<='0';b<='0';bin<='1';wait for 100 ns;
        a<='0';b<='1';bin<='0';wait for 100 ns;
        a<='0';b<='1';bin<='1';wait for 100 ns;
        a<='1';b<='0';bin<='0';wait for 100 ns;
        a<='1';b<='0';bin<='1';wait for 100 ns;
        a<='1';b<='1';bin<='0';wait for 100 ns;
        a<='1';b<='1';bin<='1';wait for 100 ns;
        wait;
    end process;
END;
```

► EXECUTION STEPS - XILINX ISE

1

New Project

→ File → New Project → full_subtractor_2hs , VHDL

2

Create half_subtractor

→ VHDL Module → half_subtractor (a,b,d,bo)

3

Enter HS code

→ `d<=a xor b; bo<=(not a) and b;`

4

Create full_subtractor

→ component half_subtractor + signals d1,bo1,bo2

5

Port maps

→ `HS1(a,b,d1,bo1); HS2(d1,bin,diff,bo2); bout<=bo1 or bo2`

6

Check Syntax

→ Synthesize-XST → 0 errors

7

Testbench + Simulate

→ All 8 combos → ISim Run 800 ns

EXAM NOTE: STRUCTURAL – full subtractor instantiates half_subtractor TWICE + OR gate.
HS borrow uses (NOT a) so a' feeds the AND. Compile HS before FS.

Full Subtractor using Half Subtractor

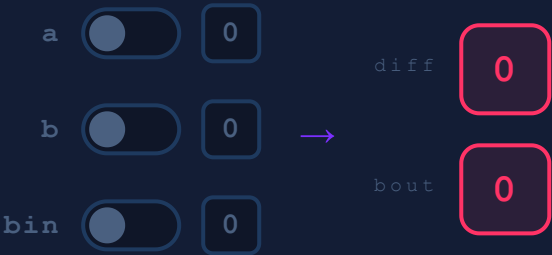
BEHAVIORAL

entity: full_sub_bh

$diff = a \oplus b \oplus bin \quad | \quad bout = a'b + a'bin + b \cdot bin$

Same full subtractor logic from the half-subtractor equations, written as a single behavioral/dataflow file – concurrent assignments, no components. diff reuses the half-subtractor XOR; bout is the sum-of-products borrow.

► LIVE SIMULATOR



$diff = a \text{ XOR } b \text{ XOR } bin \quad | \quad bout = (a' \text{ AND } b) \text{ OR } (a' \text{ AND } bin) \text{ OR } (b \text{ AND } bin)$

► TRUTH TABLE

a	b	bin	diff	bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

► VHDL - FULL_SUB_BH.VHD (BEHAVIORAL)

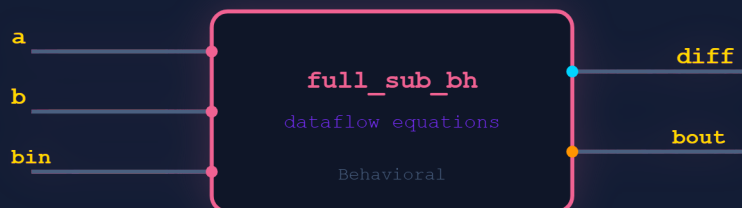
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_sub_bh is
    Port (
        a      : in  STD_LOGIC;
        b      : in  STD_LOGIC;
        bin    : in  STD_LOGIC;
        diff   : out STD_LOGIC;
        bout   : out STD_LOGIC
    );
end full_sub_bh;

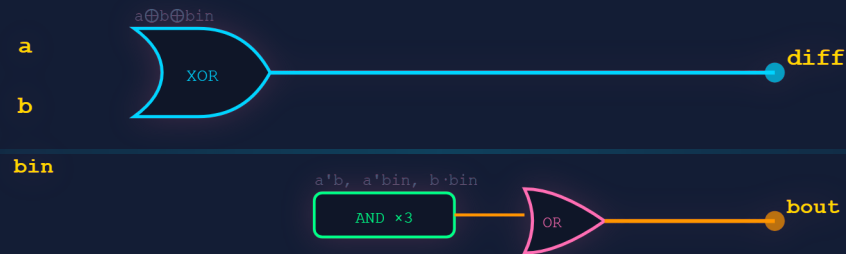
architecture Behavioral of full_sub_bh is
begin
    diff <= a xor b xor bin;
    bout <= ((not a) and b) or ((not a) and bin) or (b and bin);
end Behavioral;
```

COPY

► BLOCK DIAGRAM - FULL_SUB_BH



► GATE-LEVEL CIRCUIT



diff = 3-input XOR | bout = OR of 3 AND terms (a' inverts a)

► TESTBENCH - FULL_SUB_BH_TB.VHD

```
ENTITY full_sub_bh_tb IS END full_sub_bh_tb;
ARCHITECTURE behavior OF full_sub_bh_tb IS
    COMPONENT full_sub_bh
        PORT(a,b,bin : IN  std_logic;
             diff,bout : OUT std_logic);
    END COMPONENT;
    signal a,b,bin : std_logic := '0';
    signal diff,bout : std_logic;
BEGIN
    uut: full_sub_bh PORT MAP(a,b,bin,diff,bout);
    process begin
        a<='0';b<='0';bin<='0';wait for 100 ns;
        a<='0';b<='0';bin<='1';wait for 100 ns;
        a<='0';b<='1';bin<='0';wait for 100 ns;
        a<='0';b<='1';bin<='1';wait for 100 ns;
        a<='1';b<='0';bin<='0';wait for 100 ns;
        a<='1';b<='0';bin<='1';wait for 100 ns;
        a<='1';b<='1';bin<='0';wait for 100 ns;
        a<='1';b<='1';bin<='1';wait for 100 ns;
        wait;
    end process;
END;
```

COPY

► EXECUTION STEPS - XILINX ISE

1

New Project

→ File → New Project → full_sub_bh , VHDL

2

New Source

→ VHDL Module → full_sub_bh – ports a,b,bin,diff,bout

3

Enter Equations

→ diff<=a xor b xor bin; and bout SOP

4

Check Syntax

→ Synthesize-XST → 0 errors

5

Testbench

→ VHDL Test Bench → all 8 combos

6

Simulate

→ ISim → Run 800 ns → verify diff/bout

EXAM NOTE: BEHAVIORAL – single file, no components. bout = SOP with a' (NOT a). Same truth table as the 2-MS structural version.

8-Bit Adder

BEHAVIORAL

entity: add_8

```
s(7:0) = a(7:0) + b(7:0);
```

Adds two 8-bit unsigned numbers in a single behavioral line using vector arithmetic (IEEE.STD_LOGIC_UNSIGNED). The + operator on STD_LOGIC_VECTOR does full 8-bit ripple addition. Note: result is 8-bit, so a carry-out beyond bit 7 wraps (mod 256).

LIVE SIMULATOR

a

0

00000000

b

0

00000000

↓

+

s

0

00000000

carry-out = 0

SAMPLE RESULTS

a	b	s (dec)	s (bin)
3	5	8	00001000
15	16	31	00011111
170	85	255	11111111
200	100	44*	00101100
255	1	0*	00000000

* carry-out dropped – sum wraps mod 256 (8-bit result)

VHDL – ADD_8.VHD (BEHAVIORAL)

[COPY](#)

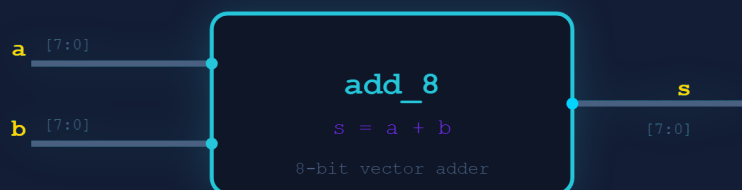
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity add_8 is
  Port (
    a : in  STD_LOGIC_VECTOR(7 downto 0);
    b : in  STD_LOGIC_VECTOR(7 downto 0);
    s : out STD_LOGIC_VECTOR(7 downto 0)
  );
end add_8;

architecture Behavioral of add_8 is
begin
  s <= a + b;
end Behavioral;
```

TIP: To keep the carry, make **s** 9 bits and write **s <= ('0'&a) + ('0'&b);**

► BLOCK DIAGRAM — ADD_8



Buses **a**, **b**, **s** are 8-bit `STD_LOGIC_VECTOR`. One + synthesizes an 8-bit ripple-carry adder.

► TESTBENCH — ADD_8_TB.VHD

[COPY](#)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

ENTITY add_8_tb IS END add_8_tb;

ARCHITECTURE behavior OF add_8_tb IS
    COMPONENT add_8
        PORT(a,b : IN  std_logic_vector(7 downto 0);
             s  : OUT std_logic_vector(7 downto 0));
    END COMPONENT;
    signal a,b,s : std_logic_vector(7 downto 0);
BEGIN
    uut: add_8 PORT MAP(a,b,s);
    process begin
        a<="00000011"; b<="00000101"; wait for 100 ns;
        a<="00001111"; b<="00010000"; wait for 100 ns;
        a<="10101010"; b<="01010101"; wait for 100 ns;
        a<="11111111"; b<="00000001"; wait for 100 ns;
        wait;
    end process;
END;
```

► EXECUTION STEPS - XILINX ISE

1

New Project

→ File → New Project → add_8 , HDL: VHDL

2

New Source

→ VHDL Module → add_8 - a,b,s as vectors (7 downto 0)

3

Add UNSIGNED library

→ use IEEE.STD_LOGIC_UNSIGNED.ALL; enables + on vectors

4

Enter code

→ s <= a + b;

5

Check Syntax

→ Synthesize-XST → 0 errors

6

Testbench + Simulate

→ Drive vector values → ISim → verify sum in binary

EXAM NOTE: The + on STD_LOGIC_VECTOR needs STD_LOGIC_UNSIGNED (or numeric_std with UNSIGNED type). Result width = operand width, so carry-out is lost unless you widen s.

D Flip-Flop

SEQUENTIAL

entity: d_ff

`on ↑clk: if r='1' → q=0, else q=d`

Positive-edge-triggered D flip-flop with synchronous reset. On each rising clock edge q captures d; if reset r is high, q clears to 0 instead. Sequential logic – uses a clocked process, not concurrent assignments.

► LIVE SIMULATOR (CLICK ↑CLK TO LATCH)



q holds its value until the next rising edge of clk

► CHARACTERISTIC TABLE

clk	r	d	q(next)	operation
↑	1	x	0	reset
↑	0	0	0	store 0
↑	0	1	1	store 1
0/1/↓	x	x	q	hold

↑ = rising edge only | x = don't care | hold = no change

► VHDL – D_FF.VHD (SEQUENTIAL)

[COPY](#)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity d_ff is
  Port (
    d    : in  STD_LOGIC;
    clk  : in  STD_LOGIC;
    r    : in  STD_LOGIC;
    q    : out STD_LOGIC
  );
end d_ff;

architecture Behavioral of d_ff is
begin
  process(clk)
  begin
    if (clk'event and clk = '1') then
      if (r = '1') then
        q <= '0';
      else
        q <= d;
      end if;
    end if;
  end process;
end Behavioral;
```

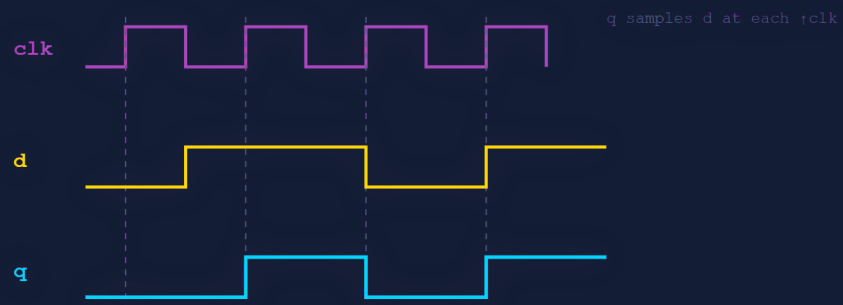
FIXED from notebook: q was inout → now out; reset branch both assigned d (did nothing) → now r='1' clears q to '0'.

► BLOCK DIAGRAM - D_FF



Inputs: d, clk, r | q = stored bit. Triangle = edge-triggered clk.

► TIMING BEHAVIOR



q changes only on rising clk edges, taking d's value at that instant.

► TESTBENCH - D_FF_TB.VHD (WITH CLOCK GEN)

[COPY](#)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

ENTITY d_ff_tb IS END d_ff_tb;

ARCHITECTURE behavior OF d_ff_tb IS
    COMPONENT d_ff
        PORT(d,clk,r : IN  std_logic;
             q       : OUT std_logic);
    END COMPONENT;
    signal d,clk,r : std_logic := '0';
    signal q : std_logic;
    constant clk_period : time := 100 ns;
BEGIN
    uut: d_ff PORT MAP(d, clk, r, q);

    clk_proc: process begin
        clk <= '0'; wait for clk_period/2;
        clk <= '1'; wait for clk_period/2;
    end process;

    stim_proc: process begin
        r <= '1'; d <= '0'; wait for 100 ns;
        r <= '0';
        d <= '1'; wait for 100 ns;
        d <= '0'; wait for 100 ns;
        d <= '1'; wait for 100 ns;
        wait;
    end process;
END;
```

► EXECUTION STEPS — XILINX ISE

1

New Project

→ File → New Project → d_ff , HDL: VHDL

2

New Source

→ VHDL Module → d_ff — ports d, clk, r, q

Write clocked process

3 → process(clk) → if clk'event and clk='1'

4

Add reset + capture

→ if r='1' q<='0' else q<=d

5

Check Syntax

→ Synthesize-XST → 0 errors

6

Testbench

→ Add clock-gen process (period 100 ns) + stimulus

7

Simulate

→ ISim → Run 500 ns → q follows d on rising edges

EXAM NOTE: SEQUENTIAL – a clocked process with `clk'event` and `clk='1'` infers a flip-flop. Output q is **out** (not inout). Testbench needs a separate clock-generator process.